

WYŻSZA SZKOŁA
INFORMATYKI i ZARZĄDZANIA
z siedzibą w Rzeszowie

KOLEGIUM INFORMATYKI STOSOWANEJ

Kierunek: INFORMATYKA

Adam Techman
Nr albumu studenta 73285

Sprawozdanie z przedmiotu „Programowanie obiektowe”

Prowadzący: mgr.inż Adrian Hałys

Laboratorium nr 1

Rzeszów 2026

Spis treści

Wstęp **4**

1 Git, C# i środowisko Visual Studio **5**
1.1 Zadanie 1 – potok pracy przy zapisywaniu zmian w Git 5
1.2 Zadanie 2 – struktura gałęzi i praca zespołowa 5
1.2.1 Struktura gałęzi 5
1.2.2 Łączenie pracy (merge) 6
1.2.3 Konflikty i ich rozwiązywanie 6
1.3 Zadanie 3 – różnice między C# a JavaScript i Python 6
1.4 Zadanie 4 – rozwiązanie a projekt w Visual Studio 7

2 Programy w języku C# **8**
2.1 Zadanie 5 – funkcja `helloWsiz()` 8
2.2 Zadanie 6 – liczby z przedziału $\langle a; b\rangle$ 8
2.3 Zadanie 7 – przybliżenie liczby π 10
2.4 Zadanie 8 – konwersja na system szesnastkowy 11

3 Debugger w MS Visual Studio **14**
3.1 Zadanie 9 – zastosowanie i funkcje debuggera 14
3.1.1 Podstawowe funkcje debuggera 14

.

Wstęp

Celem pierwszego laboratorium z przedmiotu Programowanie obiektowe było zapoznanie się z podstawowymi narzędziami pracy programisty: systemem kontroli wersji Git, środowiskiem MS Visual Studio oraz językiem C#. Zadania podzielono na dwie grupy:

- zadania teoretyczne (1–4 oraz 9) – opis pracy z repozytorium Git, porównanie języka C# z językami JavaScript i Python, różnica pomiędzy rozwiązaniem a projektem w Visual Studio oraz omówienie debuggera,
- zadania programistyczne (5–8) – proste programy konsolowe w języku C#.

Wszystkie programy zostały umieszczone w jednym pliku `lab1.cs` (przestrzeń nazw `Labolatorium_1`) Metoda `Main` wywołuje kolejno metody `Zadanie5()`, `Zadanie6()`, `Zadanie7()` i `Zadanie8()`, z których każda realizuje jedno polecenie. Program został zbudowany i uruchomiony na platformie .NET, a wyniki jego działania zamieszczono przy opisie poszczególnych zadań.

.

Rozdział 1

Git, C# i środowisko Visual Studio

1.1 Zadanie 1 – potok pracy przy zapisywaniu zmian w Git

Polecenie: Opisz potok pracy związany z zapisaniem w repozytorium Git nowych zmian w kodzie.

Zapisanie nowych zmian w repozytorium Git składa się z trzech głównych kroków. Pliki przechodzą kolejno przez katalog roboczy, poczekalnię (*staging area*), lokalne repozytorium i na końcu trafiają do repozytorium zdalnego.

1. `git add <nazwa_pliku>` (lub `git add .` dla wszystkich plików) – dodanie zmodyfikowanych plików do poczekalni. W ten sposób deklaruje się, co ma wejść w skład nowej wersji.

2. `git commit -m "opis zmian"` – trwałe zapisanie zmian w lokalnym repozytorium na komputerze wraz z komentarzem opisującym, co zostało zrobione.

3. `git push` – wysłanie lokalnych zapisów (commitów) na serwer zdalny (np. GitHub lub GitLab), aby inni mieli do nich dostęp.

Pomocniczo stosuje się polecenie `git status`, które pokazuje, które pliki zostały zmienione i które znajdują się już w poczekalni, oraz `git pull`, które przed wysłaniem zmian pobiera najnowszą wersję kodu z serwera.

1.2 Zadanie 2 – struktura gałęzi i praca zespołowa

Polecenie: Opisz jak powinna wyglądać struktura gałęzi na repozytorium Git w przypadku równoległej pracy kilku osób. Opisz jak łączyć efekty pracy kilku osób i czym są oraz jak rozwiązywać konflikty w kodzie.

1.2.1 Struktura gałęzi

Zazwyczaj w repozytorium istnieje gałąź główna (np. `main` lub `master`) zawierająca stabilny, działający kod. Każdy programista tworzy własną gałąź roboczą (tzw. *feature branch*, np. `feature/logowanie`) odgałęzioną od `main`, aby pracować nad swoim zadaniem bez psucia kodu innym osobom.

1.2.2 Łączenie pracy (merge)

Gdy praca na gałęzi roboczej jest skończona, łączy się ją z gałęzią `main` za pomocą polecenia `git merge` lub – częściej w firmach – poprzez *Pull Request*. Pull Request pozwala innym członkom zespołu sprawdzić kod (*code review*) przed jego scaleniem.

1.2.3 Konflikty i ich rozwiązywanie

Konflikty powstają, gdy dwie osoby zmienią dokładnie ten sam fragment kodu i Git nie wie, którą wersję zachować. W takiej sytuacji Git wstawia do pliku znaczniki oddzielające obie wersje:

```
<<<<<<< HEAD
wersja z bieżącej gałęzi
=======
wersja z łączonej gałęzi
>>>>>>> feature/logowanie
```

Aby rozwiązać konflikt, należy:

1. otworzyć plik z konfliktem (edytor podświetli sporne miejsca),
2. ręcznie usunąć znaczniki dodane przez Gita i zostawić właściwy kod,
3. zapisać plik i wykonać `git add`,
4. wykonać `git commit`, aby zatwierdzić ostateczną wersję.

1.3 Zadanie 3 – różnice między C# a JavaScript i Python

Polecenie: Opisz, czym technicznie różni się język C# od poznawanych na wcześniejszych zajęciach językach programowania – JavaScript'u i Python'a.

Kompilacja a interpretacja. C# jest językiem kompilowanym – przed uruchomieniem kod musi zostać przetłumaczony przez kompilator na język pośredni (IL), który następnie jest wykonywany przez środowisko uruchomieniowe .NET (CLR). JavaScript i Python to języki interpretowane – kod jest wykonywany przez interpreter w trakcie działania, bez osobnego etapu kompilacji.

Typowanie statyczne a dynamiczne. C# jest językiem silnie i statycznie typowanym – typ zmiennej (np. `int` lub `string`) jest znany już podczas kompilacji, a błędy typów wyłapywane są przed uruchomieniem programu. JavaScript i Python dobierają typy dynamicznie w trakcie działania programu.

Paradygmat. C# jest od podstaw językiem obiektowym – kod umieszcza się wewnątrz klas. JavaScript i Python pozwalają na luźniejsze pisanie kodu, np. w postaci swobodnych skryptów bez tworzenia klas.

Tabela 1.1: Porównanie języków C#, JavaScript i Python

Cecha	C#	JavaScript	Python
Sposób wykonania	kompilacja do IL	interpretacja	interpretacja
Typowanie	statyczne	dynamiczne	dynamiczne
Kod poza klasami	ograniczony	tak	tak

1.4 Zadanie 4 – rozwiązanie a projekt w Visual Studio

Polecenie: Czym w MS Visual Studio różni się rozwiązanie od projektu?

Projekt (`.csproj`) zawiera kod źródłowy, zasoby i ustawienia potrzebne do zbudowania pojedynczego pliku wynikowego – np. jednej aplikacji wykonywalnej (`.exe`) lub jednej biblioteki (`.dll`).

Rozwiązanie (`.sln`) to kontener, który grupuje jeden lub więcej logicznie powiązanych projektów. Pozwala otworzyć i zarządzać całym systemem naraz – np. jedno rozwiązanie może zawierać aplikację webową, osobny projekt z testami i osobny projekt obsługujący bazę danych.

.

Rozdział 2

Programy w języku C#

2.1 Zadanie 5 – funkcja `helloWsiz()`

Polecenie: Napisz w C# program zawierający bezparametrową funkcję `helloWsiz()` zwracającą łańcuch tekstowy o treści „Hello WSIiZ”. Wypisz w konsoli wynik działania tej funkcji.

Funkcja `helloWsiz()` zwraca stały napis, a metoda `Zadanie5()` wypisuje go w konsoli za pomocą `Console.WriteLine`.

Listing 2.1: Rozwiązanie zadania 5

```
static string helloWsiz()
{
    return "Hello WSIiZ";
}

static void Zadanie5()
{
    Console.WriteLine(helloWsiz());
}
```

Wynik działania:

```
Hello WSIiZ
```

2.2 Zadanie 6 – liczby z przedziału $\langle a; b \rangle$

Polecenie: Napisz w C# program, który zapyta użytkownika o dwie liczby całkowite a oraz b i wypisze w konsoli wszystkie liczby z przedziału zamkniętego $\langle a; b \rangle$.

Do wczytywania liczb służy pomocnicza funkcja `WczytajInt`, która powtarza pytanie tak długo, aż użytkownik poda poprawną liczbę całkowitą (sprawdzenie przez `int.TryParse`). Funkcja ta jest wykorzystywana również w zadaniu 7. Jeżeli użytkownik poda $a > b$, wartości zostają zamienione miejscami, dzięki czemu przedział jest zawsze wypisywany rosnąco. Zmienna sterująca pętli ma typ `long`, co zapobiega przepełnieniu, gdy b jest równe `int.MaxValue`.

Listing 2.2: Rozwiązanie zadania 6

```
static void Zadanie6()
{
    int a = WczytajInt("Podaj liczbę całkowitą a: ");
    int b = WczytajInt("Podaj liczbę całkowitą b: ");
    WypiszPrzedzial(a, b);
}

static void WypiszPrzedzial(int a, int b)
{
    if (a > b)
    {
        (a, b) = (b, a);
    }

    for (long i = a; i <= b; i++)
    {
        Console.Write(i);
        Console.Write(i < b ? " " : Environment.NewLine);
    }
}

static int WczytajInt(string komunikat)
{
    while (true)
    {
        Console.Write(komunikat);
        string? wejscie = Console.ReadLine();
        if (wejscie == null)
        {
            throw new EndOfStreamException("Brak danych wejściowych.");
        }
        if (int.TryParse(wejscie, out int wynik))
        {
            return wynik;
        }
        Console.WriteLine("To nie jest poprawna liczba całkowita, spróbuj
            ponownie.");
    }
}
```

Wynik działania dla $a = 3, b = -2$ (zamiana kolejności):

```
Podaj liczbę całkowitą a: 3
Podaj liczbę całkowitą b: -2
-2 -1 0 1 2 3
```

Wynik działania z błędnymi danymi wejściowymi:

```
Podaj liczbę całkowitą a: 1
```

```
Podaj liczbę całkowitą b: abc
To nie jest poprawna liczba całkowita, spróbuj ponownie.
Podaj liczbę całkowitą b: 4
1 2 3 4
```

2.3 Zadanie 7 – przybliżenie liczby π

Polecenie: Napisz w C# i przetestuj funkcję, która obliczy wartość n-elementowego ciągu opisanego wzorem

$$y = 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots\right)$$

i wypisze ją w konsoli.

Podany ciąg to szereg Leibniza, którego suma dąży do liczby π. Dla n wyrazów można go zapisać jako

$$y_n = 4\sum_{k=0}^{n-1} \frac{(-1)^k}{2k+1}. \tag{2.1}$$

Funkcja `PrzyblizeniePi` w pętli sumuje kolejne wyrazy $\frac{1}{2k+1}$, dodając je dla parzystych k i odejmując dla nieparzystych, a na końcu mnoży sumę przez 4. Metoda `Zadanie7()` wczytuje $n \geq 1$, wypisuje wynik, wartość `Math.PI` oraz błąd bezwzględny, a następnie testuje zbieżność dla kilku wartości n.

Listing 2.3: Rozwiązanie zadania 7

```
1  static double PrzyblizeniePi(int n)
2  {
3      double suma = 0.0;
4      for (int k = 0; k < n; k++)
5      {
6          double wyraz = 1.0 / (2 * k + 1);
7          suma += (k % 2 == 0) ? wyraz : -wyraz;
8      }
9      return 4 * suma;
10 }
11
12 static void Zadanie7()
13 {
14     int n;
15     do
16     {
17         n = WczytajInt("Podaj liczbę wyrazów n (n >= 1): ");
18     } while (n < 1);
19
20     double y = PrzyblizeniePi(n);
21     Console.WriteLine($"Przybliżenie dla n = {n}: {y}");
22     Console.WriteLine($"Math.PI:                {Math.PI}");
23     Console.WriteLine($"Błąd:                   {Math.Abs(Math.PI - y)}");
24
```

```
    Console.WriteLine();
    Console.WriteLine("Test zbieżności:");
    foreach (int liczba in new[] { 1, 2, 10, 100, 1000, 1_000_000 })
    {
        double wynik = PrzyblizeniePi(liczba);
        Console.WriteLine($"  n = {liczba,9}: y = {wynik:F10}, błąd = {Math
            .Abs(Math.PI - wynik):E2}");
    }
}
```

Wynik działania dla $n = 1000$:

```
Podaj liczbę wyrazów n (n >= 1): 1000
Przybliżenie dla n = 1000: 3,140592653839794
Math.PI:                   3,141592653589793
Błąd:                      0,000999999749998981

Test zbieżności:
  n =         1: y = 4,0000000000, błąd = 8,58E-001
  n =         2: y = 2,6666666667, błąd = 4,75E-001
  n =        10: y = 3,0418396189, błąd = 9,98E-002
  n =       100: y = 3,1315929036, błąd = 1,00E-002
  n =      1000: y = 3,1405926538, błąd = 1,00E-003
  n =   1000000: y = 3,1415916536, błąd = 1,00E-006
```

Test pokazuje, że szereg jest zbieżny do π, ale bardzo wolno – błąd jest w przybliżeniu równy $1/n$, więc każde dziesięciokrotne zwiększenie liczby wyrazów daje tylko jedną dodatkową poprawną cyfrę.

2.4 Zadanie 8 – konwersja na system szesnastkowy

Polecenie: Napisz funkcję, która dla liczby dziesiętnej podanej jako parametr wypisze w konsoli jej wartość w systemie szesnastkowym. Nie używaj wbudowanych funkcji do konwersji liczb. Wynikową liczbę potraktuj jako napis.

Konwersja wykorzystuje metodę kolejnych dzieleń przez 16. Reszta z dzielenia (0–15) wskazuje cyfrę w napisie `"0123456789ABCDEF"`, która jest dopisywana na początek wyniku, a liczba jest dzielona całkowicie przez 16 aż do osiągnięcia zera. Dla liczb ujemnych konwertowana jest wartość bezwzględna i dopisywany jest znak minus. Wartość bezwzględna jest obliczana jako `-(liczba + 1) + 1` na typie `ulong`, dzięki czemu poprawnie obsłużona jest również najmniejsza liczba typu `long`, której wartości bezwzględnej nie da się zapisać w typie `long`.

Przykład dla liczby 2026:

$$\begin{aligned} 2026 : 16 &= 126 \quad \text{reszta } 10 \to \text{A} \\ 126 : 16 &= 7 \quad \text{reszta } 14 \to \text{E} \\ 7 : 16 &= 0 \quad \text{reszta } 7 \to 7 \end{aligned}$$

Odczytując reszty od końca otrzymujemy $2026_{10} = 7\text{EA}_{16}$.

Listing 2.4: Rozwiązanie zadania 8

```
static string NaSzesnastkowy(long liczba)
{
    const string cyfry = "0123456789ABCDEF";
    if (liczba == 0)
    {
        return "0";
    }
    bool ujemna = liczba < 0;
    ulong wartosc = ujemna ? (ulong)(-(liczba + 1)) + 1 : (ulong)liczba;

    string wynik = "";
    while (wartosc > 0)
    {
        int reszta = (int)(wartosc % 16);
        wynik = cyfry[reszta] + wynik;
        wartosc /= 16;
    }

    return ujemna ? "-" + wynik : wynik;
}

static void WypiszSzesnastkowo(long liczba)
{
    Console.WriteLine($"{liczba} (dec) = {NaSzesnastkowy(liczba)} (hex)");
}

static void Zadanie8()
{
    long liczba;
    while (true)
    {
        Console.Write("Podaj liczbę dziesiętną (całkowitą): ");
        string? wejscie = Console.ReadLine();
        if (wejscie == null)
        {
            return;
        }
        if (long.TryParse(wejscie, out liczba))
        {
            break;
        }
        Console.WriteLine("To nie jest poprawna liczba całkowita, spróbuj
            ponownie.");
    }
    WypiszSzesnastkowo(liczba);
}
```

Wyniki działania dla przykładowych danych:

```
Podaj liczbę dziesiętną (całkowitą): 255
255 (dec) = FF (hex)

Podaj liczbę dziesiętną (całkowitą): -255
-255 (dec) = -FF (hex)
```

.

Rozdział 3

Debugger w MS Visual Studio

3.1 Zadanie 9 – zastosowanie i funkcje debuggera

Polecenie: Opisz do czego służy Debugger w MS Visual Studio i jakie są jego podstawowe funkcje.

Debugger w MS Visual Studio to narzędzie służące do wykrywania i naprawiania błędów w programie. Pozwala sprawdzić, jak program działa krok po kroku, oraz znaleźć przyczynę jego nieprawidłowego działania.

3.1.1 Podstawowe funkcje debuggera

Breakpoint (punkt przerwania) – zatrzymuje program w wybranym miejscu.

Step Over – wykonuje kolejną instrukcję bez wchodzenia do wywoływanej funkcji.

Step Into – pozwala wejść do funkcji i sprawdzić jej działanie.

Step Out – kończy wykonywanie aktualnej funkcji i wraca do miejsca, z którego ją wywołano.

Podgląd zmiennych (okna *Locals*, *Autos*, *Watch*) – umożliwia sprawdzanie aktualnych wartości zmiennych.

Call Stack (stos wywołań) – pokazuje, jakie funkcje zostały wywołane i w jakiej kolejności.

Continue – wznawia działanie programu po jego zatrzymaniu.

Debugger przydaje się np. przy sprawdzaniu funkcji `PrzyblizeniePi` z zadania 7 – po ustawieniu punktu przerwania wewnątrz pętli można krok po kroku obserwować, jak zmieniają się wartości zmiennych `k`, `wyraz` i `suma`.